
Structural Coverage: Do AI Provers Use Mathlib the Way Mathematicians Do?

Anirudh Balaji

PES University, Bengaluru, India
PES1UG25AM042@stu.pes.edu

Abstract

A Lean proof is not only a binary success. It is a selection of premises from a library, and that selection induces a subgraph of mathlib4’s dependency graph. Two proofs can both be correct and induce structurally different subgraphs; `pass@k` is blind to the difference. We define structural coverage over these subgraphs and compare 29,750 released machine Lean proofs against 70,086 human mathlib declarations. Three findings. (i) Machine proofs draw on a small subset of the human premise vocabulary, one to two orders of magnitude smaller: 78,642 against 1,127 premises, Jaccard 0.0129, leaving 98.7% of the human vocabulary untouched. The gap survives every control we build, narrowing to $19\times$ – $28\times$ when the human corpus is restricted to the regions the machine corpus occupies and to $5.7\times$ under an exploratory genre-matched comparison against mathlib’s own competition archive. (ii) The machine corpus concentrates on a stratum that is essentially absent from mathlib: arithmetic-tactic proofs of at most two tactic steps are 23.1% of the machine corpus and 0.02% of mathlib, and a single premise, `sq_nonneg`, accounts for 27.0% of all machine premise occurrences. (iii) A cautionary negative result: the graph’s published depth attribute proved to be a centrality proxy, not a derivation-length measure, and we withdraw a finding built on it. Structural coverage is a non-redundant evaluation axis, cheap to compute from released corpora.

1 Introduction

The MATH-AI call asks: “*Humans vs. machines: What are the comparative strengths, limitations, and characteristic failure modes of human mathematicians and AI systems?*” Formal theorem proving offers an unusually direct way to answer it. When a prover closes a goal in Lean, it does not merely produce a truth value. It selects premises from mathlib4, and that selection induces a subgraph of the library’s dependency graph.¹ Two proofs of the same statement can both compile and still induce subgraphs that share almost nothing. Standard evaluation cannot see this. `pass@k` counts closures; it is by construction blind to which part of the library was used to get there.

We ask whether machine-generated proofs use mathlib the way human contributors do, and answer by measurement. Our object of study is the *explicit subgraph* of mathlib4’s dependency graph [Li et al., 2026] — the source-visible portion approximating human-intended dependencies — and our comparison is between the premise vocabularies human and machine proofs induce over it. Three findings follow.

1. Vocabulary coverage differs by one to two orders of magnitude, and the gap is not an artifact of corpus size or library area. Across full corpora the human premise vocabulary is 78,642

¹All metrics come from committed CSVs produced by scripts with pinned data revisions and fixed seeds. Code, the committed CSVs, the pinned revisions and the seeds will be released.

against the machine corpus’s 1,127, a ratio of $69.8\times$, with Jaccard 0.0129 and 98.7% of the human vocabulary never touched by any machine proof. Every control we build narrows the gap and none closes it: rarefaction to equal n leaves $44.0\times$, a 1:1 class- and step-matched design $18.8\times$, and restriction to the 42 library regions the machine vocabulary inhabits $28.5\times$.

2. There is a characteristic concentration, and it is a stratum. Proofs classified as arithmetic-tactic with at most two tactic steps are 23.1% of the machine corpus and 0.02% of mathlib, or 14 of 70,086 declarations. Machine premise usage is correspondingly concentrated: `sq_nonneg` alone is 27.0% of machine premise occurrences and the top 25 premises are 68.1%.

3. A negative methodological result about the released graph itself. We built a depth metric on the graph release’s published per-declaration layer attribute, promoted the resulting gap to a stated counter-finding, and then found the attribute to be a centrality proxy. We withdraw the finding and report the failure, because it is a second independent instance of a caution the graph’s own authors raise [Li et al., 2026] and because a source-visible pipeline can consume such a column for weeks without any signal that it is wrong.

2 Setup

The graph and the explicit subgraph We use the published mathlib4 dependency graph of Li et al. [2026] (308,129 declarations, 8,436,366 edges) at a pinned revision, built on mathlib4 at commit 534cf0b. We do not rebuild it. That work reports that 74.2% of dependency edges are compiler-synthesized and invisible in source, and defines an *explicit subgraph* as a closer proxy for human-intended dependencies. The explicit subgraph, 2,178,534 edges, is our object of study, and source-visible extraction is the appropriate instrument for it.

Corpora Our human reference is the 70,086 mathlib declarations we can locate in source and extract premises from. Our machine corpus is Goedel-LM/Lean-workbook-proofs, 29,750 released proofs of Lean Workbook problems, pre-verified by its releasers, so no Lean execution is needed for correctness. Both are censuses, not samples.

The confound we cannot remove The machine corpus targets competition statements while mathlib targets library development. No control available to us removes this confound on problem selection; Section 5 states what it permits us to claim. The comparison is over the shared premise vocabulary.

3 Method

Extraction We resolve identifier tokens in a declaration’s proof body against the graph’s declaration names at declaration scope, comments and strings stripped, by exact name or by completion under a namespace prefix in scope; bare dot-notation residue (`.mp`) is kept separate, as in Ji [2026].

Retention, and why we report it rather than gate on it Against explicit-subgraph ground truth on the mathematical hub layer the extractor recovers 0.6124 of targets. Scope decomposes that figure. Matching premise names anywhere in the whole source file recovers only 0.8193 of mathematical-layer explicit edges, so an absolute target near 1 is unreachable by construction: that ceiling belongs to Lean’s elaborator, not to our parser. Nor is it the bound the extractor works against. At declaration scope, where it runs, the reachable bound is 0.6948, so efficiency is $0.6124/0.6948 = \mathbf{0.8815}$; dividing by the whole-file 0.8193 instead gives 0.7475, against a scope the extractor never sees. We report retention with this decomposition, having mis-specified a pass/fail threshold twice.

The hub-layer partition Results below split premises into language-infrastructure and mathematical-content layers. This partition is ours and provenance-based: a declaration is infrastructure if its `file_module` lies outside Mathlib, if it is an instance or coercion, if its `kind` is a kernel kind, or if its name carries a generated-name marker. It is unrelated to the per-declaration layer attribute withdrawn in Section 5.

Population neutrality What matters is not how much the instrument misses but whether it misses differentially across corpora. Per-class efficiency against the reachable bound is flat across the five

design	human	machine	ratio	Jaccard	human-only
full corpora	78,642	1,127	69.8×	0.0129	0.9871
rarefied to machine n	49,601	1,127	44.0×	—	—
area-restricted (primary control)	32,091	1,127	28.5×	0.0315	0.9684
1:1 class- and step-matched	21,162	1,123	18.8×	0.0360	0.9634
genre-matched (exploratory)	1,101	193	5.7×	—	—

Table 1: Premise vocabulary under five designs. The area-restricted row keeps the 42 regions the machine vocabulary occupies (40.8% of human premises, no machine loss). The genre-matched row is exploratory; see Section 5.

tactic classes (0.874–0.896), so the instrument does not preferentially penalise either corpus’s tactic mix. That flatness is what carries the claim, and we rest it on nothing else.

Length control Short proofs mechanically use fewer, commoner premises, so comparisons are length-controlled on *tactic steps*, not characters: the corpora overlap 0.856 on steps against 0.215 on characters, so a character axis is largely a formatting control on hand-wrapped mathlib source. Coverage needs no such standardization: the effect lies orders of magnitude outside the regime a length correction operates in, so no reweighting available to us could produce or remove it. Corpus size is handled by design instead, in the rarefied and matched rows of Table 1.

4 Results

4.1 Vocabulary coverage

Table 1 gives the primary result under five designs of increasing severity. Two one-sided extractor artifacts — a declaration resolving its own name, and the `lemma` keyword resolving to a real mathlib declaration — inflated the human side only. They account for 33.6% of the uncorrected human count, and are removed throughout, taking the headline from 118,514 to 78,642 human premises before any comparison is made.

The corpus-size objection is answered by measurement. Rarefying the human corpus to the machine corpus’s proof count still gives 49,601 premises (44.0× at equal n), and the 1:1 design matched on tactic class and step count gives 18.8× with 96.3% untouched.

The area-restricted design is our primary control, its rule fixed before it ran: every top-level region containing at least one machine premise. The rule is support-based, so there is no threshold or top- k to tune. The machine vocabulary occupies 42 of the 1,595 regions the human vocabulary spans; those hold 40.8% of human premises; inside them the human vocabulary is still 28.5× larger, 96.8% untouched. A tighter twelve-namespace variant gives 19.2× but discards 31.0% of the machine vocabulary, so we report the range 19×–28×.

4.2 A characteristic concentration

The machine corpus concentrates on a stratum essentially absent from mathlib. Proofs with an arithmetic-flavoured tactic set (`nlinarith`, `linarith`, `positivity`, ...) and at most two tactic steps are 23.1% of the machine corpus (6,869 of 29,750) and 0.02% of mathlib, or 14 of 70,086 declarations. We report this as a result: it is the largest cell of the machine corpus and has no human counterpart. Usage is concentrated to match. `sq_nonneg` alone is 27.0% of machine premise occurrences and the top 25 are 68.1%, and machine proofs resolve 2.08 premises per proof against 6.24 for human declarations. The composition of that vocabulary differs too: 25.6% of the machine premise vocabulary sits in the language-infrastructure hub layer against 4.6% of the human one, 5.6× more infrastructure-weighted. That is consistent with the single-step automation stratum above: proofs closed by a single arithmetic tactic draw disproportionately on infrastructure rather than mathematical content.

4.3 Structural breadth

Breadth counts distinct Louvain communities touched per tactic step. Its sign is stable and negative: machine proofs touch fewer regions per step at every scope. Its magnitude is design-dependent by $4\times$, at -0.607 full corpora, -0.780 common support, -0.201 matched 1:1. We quote all three, never one alone. The partition is also coarse here (median community size 1; the five largest hold 68% of the graph), so breadth is a secondary characterization with its limitation attached, not a load-bearing result.

What this buys an evaluation Coverage separates proof sets that `pass@k` cannot, needs no standardization or imputation, and is computable from any released corpus against a published graph. Its Jaccard holds at 0.0129 on full corpora and 0.0128 under common support, the restriction that reverses or destroys every other comparison here; the 1:1 matched design moves it to 0.0360.

5 Limitations

Genre, and the control we do not have Area restriction fixes library *area* but not *genre*: a mathlib theorem is general-purpose infrastructure, a competition problem a self-contained puzzle. The proper control is a same-problem comparison and it is **not currently available**: the only Lean 4 human miniF2F [Zheng et al., 2022] proofs in existence are 67, and they are a prover’s own few-shot prompt pool; the standard Lean 4 port’s 488 statements are all sorry, and the original release’s 155 proofs are Lean 3. As an exploratory substitute, mathlib’s Archive/Imo (508 human declarations, no sorry) draws 1,101 premises against 193 for the machine corpus rarefied to 508 proofs — $5.7\times$. That figure is not pre-registered and is a lower bound: 467 of the 508 cite local lemmas the extractor cannot see, Archive lying outside the graph’s build target, which also leaves it not extraction-audited.

Instrument Retention is 0.6124. We argue population neutrality from flat per-class efficiency rather than assume it, and under-recovery is conservative in direction, but the vocabularies are instrument-filtered and should be read so. Three further choices run the same way: `symm/trans` are left unexcluded though they resolve 5,222 times on the human side and 14 on the machine one, prefix resolution is uncorrected though it supplies 3.657 premises per proof to the human side against 0.374 to the machine one, and coverage is reported over both hub layers, where the mathematical layer alone would give $89.6\times$ against the reported $69.8\times$. All three leave the reported gap smaller than a stricter accounting would give.

Depth is withdrawn We built a depth metric on the release’s per-declaration layer attribute, which we took to be distance from primitives. It is not, and the error was ours: its lowest layer holds 119,633 nodes of maximum in-degree 0, uncited leaves rather than axioms, its highest holds `Eq.ref1`, `Set` and `LE.le`, and it correlates $+0.677$ with `log in-degree`. That is what the attribute should look like if edges run declaration-to-premise with sources at level zero, so it is consistent with the release’s definition; we read it backwards. As we use it the quantity is a centrality proxy, so a gap in it cannot carry a depth claim, which is the inference Li et al. [2026] rule out. We withdraw rather than reinterpret it, since re-pointing a quantity at a new claim after its comparison has been seen is a researcher degree of freedom. The check is cheap: correlate a derived attribute against in-degree and inspect its extremes.

Scope One machine corpus. These are comparisons of two distributions, not causal claims: we write “machine proofs concentrate on,” never “provers are biased toward.”

6 Related work

Li et al. [2026] give the multilayer analysis of mathlib4 we build on, release the graph, and pose structural drift from AI proofs as future work; we extend their human baseline to the machine population. Pham et al. [2026] find provers biased toward easy subtheorems and Petrov et al. [2026] assess proof quality in ProofRank, both at theorem or whole-proof level, where we work on the induced premise subgraph. Xin et al. [2026] correlate success with transitive dependency closure size, which counts how much a proof depends on, not where in the library it sits.

References

- Xinze Li, Nanyun Peng, Simone Severini, and Patrick Shafto. The Network Structure of Mathlib. *arXiv:2604.24797*, 2026.
- QuocViet Pham, Elvir Karimov, Andrey Galichin, and Ivan Oseledets. TheoremBench: Evaluating LLMs on Theorem Proving in Formal Mathematics. *arXiv:2606.09450*, 2026.
- Ivo Petrov, Jasper Dekoninck, Dimitar I. Dimitrov, and Martin Vechev. Not All Proofs Are Equal: Evaluating LLM Proof Quality Beyond Correctness. *arXiv:2605.10379*, 2026. ICML 2026 AI for Math workshop.
- Yutong Xin, Qiaochu Chen, Greg Durrett, and Işıl Dillig. VeriSoftBench: Repository-Scale Formal Verification Benchmarks for Lean. *arXiv:2602.18307*, 2026.
- Junye Ji. CSLibPremiseBench: Structure-Guided Premise Retrieval and Label Robustness for Lean 4 Computer-Science Theorems. *arXiv:2605.14549*, 2026.
- Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: A Cross-System Benchmark for Formal Olympiad-Level Mathematics. *ICLR*, 2022.